

Training on Data Analysis Reproducibility via Containerization with Apptainer

Roy Cruz Candelaria¹, Wouter Deconinck², Aman Desai³, Guillermo Fidalgo Rodríguez⁴, Michel Hernandez Villanueva⁵, Kilian Lieret⁶, Valeriia Lukashenko⁷, Sudhir Malik⁸, Marco Mambelli⁹, Tetiana Mazurets⁸, Alexander Moreno Briceño¹⁰, Andres Rios-Tascon⁶, and Richa Sharma⁸

¹ University of Wisconsin-Madison ² University of Manitoba ³ Adelaide University ⁴ University of Alabama
⁵ Brookhaven National Laboratory ⁶ Princeton University ⁷ University of Zurich ⁸ University of Puerto Rico
⁹ Rico Mayaguez ¹⁰ Fermi National Accelerator Laboratory ¹⁰ Universidad Antonio Nariño

DOI: 10.xxxxxx/draft

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Submitted: 13 March 2026

Published: unpublished

License

Authors of papers retain copyright¹⁷ and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))²⁰

Summary

We present the material and resources developed for training physicists on containerization technologies enabled by Apptainer. In the context of analysis preservation using Apptainer's capabilities, we have developed examples that execute common tools in High Energy Physics (HEP) and Nuclear Physics within containers. Training physicists on containerization technologies is of utmost importance in today's research landscape. By embracing these technologies, users can achieve enhanced reproducibility, portability, collaboration, and resource efficiency, assuring the conditions and integrity of the scientific analysis process. This training module, "[Introduction to Apptainer/Singularity](#)", is part of the HEP Software Foundation Training Center, which aims to equip newcomers to the field of High Energy Physics with the necessary software skills and best practices.

Statement of Need

The high complexity of the HEP analyses creates major challenges in terms of capturing and preserving the analysis and the knowledge surrounding it. In addition, the high value of the data produced by the experiments demands having a framework in place for data and knowledge preservation that allows reuse, reinterpretation, and reproducibility of research outcomes. Containerization plays a crucial role in modern software development and scientific research. Containers encapsulate the entire environment, including dependencies, libraries, and configurations, ensuring that the software or research environment can be replicated accurately across different systems. This reproducibility eliminates the infamous "works on my machine" problem and enables others to reproduce results and build upon existing work easily. Therefore, concepts such as reproducibility, preservation, and distribution are enabled by containerization.

Within the wide range of solutions available, Apptainer, formerly known as Singularity ([Kurtzer, 2017](#)), is a container platform designed by and for scientists. It runs across various Linux operating systems and computing environments without special requirements or superuser permissions, simplifying deployment across different cloud platforms, clusters, and High-Performance Computing (HPC) systems, thereby enabling efficient use of computing resources and reducing the time and effort required to scale analysis workflows. Apptainer has been designed to use single-file-based container images, facilitating distribution, archiving, and sharing. The containers can run as a regular

application, simplifying the integration with resource managers and distributed computing environments. Additionally, the containers preserve the permissions in the environment: the user outside the container can be the same user inside, preventing any security concerns from system administrators.

By training collaborators on Apptainer, experiments can establish a common platform and language for sharing and collaborating on analysis projects. Containers serve as self-contained units that can be easily shared, enabling the reproducibility of results across teams, institutions, and even globally when providing open data sets accessible to everyone. The preservation of the analysis is ensured for years to come, independently of the availability of dependencies for the analysis, as long as Apptainer is supported in future operating system versions. Therefore, it is of the utmost importance to train individuals in containerization technologies due to the numerous benefits they offer, such as enhanced reproducibility and seamless portability.

Curriculum

Learning objectives

The training module covers basic concepts of containerization on Apptainer. Students learn about images and containers, how they differ from virtual machines, and the design goals behind Apptainer to develop reproducible environments that can be executed on different platforms in the context of analysis preservation.

Specific goals for this training module include how to:

- Pull Apptainer images from libraries
- Run commands inside Apptainer containers
- Build Apptainer containers with user requirements and from a single file definition
- Share files from the host system to the Apptainer container and vice versa

Prerequisites

Students are required to have a basic knowledge of the Unix Shell commands. In addition, they must have either access to a computing system with Apptainer available, such as an institutional cluster, or install Apptainer locally on a Linux system, on a Mac using Lima and Qemu, or in WSL on Windows machines. The HEP Software Foundation (HSF), a global community that facilitates collaboration and common efforts in the development and sustainability of software for high-energy physics (Albrecht & others, 2019), provides in its [training center](#) the basic material to cover the prerequisites listed above. The Setup section in the training module covers Apptainer's installation on the different platforms.

Contents

The training module (or lesson) is divided into episodes (individual chapters). Every episode has stated learning objectives, a main body that includes exercises with solutions, and a summary in key points to ensure the accomplishment of the learning objective. These are the episodes with their corresponding specific objectives:

- **Introduction:** Learning the containerization concepts and design goals behind Apptainer.
- **Containers and Images:** Learning to search and pull images from the Singularity library and Docker Hub, and interacting with the containers using the command line interface.
- **Building Containers:** Downloading, assembling, and modifying containers from available images in the repositories.
- **Containers from Definition Files:** Creating a container from a definition file.

- **Sharing Files between Host and Container:** Mapping directories on your host system to directories within your container and learning about the bind paths included automatically in all containers.
- **Apptainer Instances:** Running containers in a detached mode to keep services up and deploying instances via definition files.
- **Bonus Episode: Building and Deploying an Apptainer Container to GitHub Packages:** Using GitHub actions to build an Apptainer container and share it via GitHub Container Registry (GHCR).

The examples and exercises use ROOT ([Antcheva & others, 2009](#)) and Python, which are widely used in the HEP and Nuclear Physics communities. Students practice the usage of containers to run ROOT interactively through PyROOT ([Galli et al., 2020](#)), building containers with the Pythia8 physics event generator ([Bierlich & others, 2022](#)) and the Uproot library ([Pivarski & others, 2017](#)) for reading ROOT files, and creating definition files that execute RooFit tutorials ([Verkerke & Kirkby, 2003](#)). In later episodes, participants deploy long-running services such as a Jupyter notebook server with ROOT support and learn to share data between the host and the container using bind mounts.

The training module is designed to be used as an aide by instructors teaching live or to be used by students learning independently. To facilitate asynchronous learning, each episode also includes a recording where the instructor explains the material and guides students through the exercises, similar to a live class.

Teaching Experience

This module on Apptainer has been used during the HSF & IRIS-HEP ([IRIS-HEP, n.d.](#)) training events on [Analysis Reproducibility](#) (formerly known as [Analysis Preservation](#) and [Analysis Pipelines](#)). During a week, we covered tools that help the participants to integrate containerization into their scientific workflows for enhanced reproducibility. Our focus was to guide researchers with strong backgrounds in data analysis through the practical aspects of using Apptainer to encapsulate their analysis pipeline. Participants learned the basic concepts of containerization and how they can be applied in the context of data-intensive workflows.

Through pre-recorded lectures and hands-on exercises, participants experience firsthand how to build and run Apptainer containers, convert existing Docker images into Apptainer for HPC environments, and manage software dependencies for long-term analysis preservation, all with resources they are familiar with. During the live mentoring sessions and support via Slack, we helped participants troubleshoot setup issues, review exercises, and understand how they can apply these tools to their own analysis. This interactive component was essential for tailoring our advice to the computing environment available to the participants.

To collect metrics related to the overall learning experience, we have implemented a two-step survey process, circulated among participants before and after the training events. The pre-event feedback enables educators to tailor their approach to meet the unique needs of each group, ensuring that attendees receive explanations at a proper level of complexity. The post-event surveys assess the effectiveness of the training event and gather valuable information on what worked well and what parts require revision. This feedback loop is fundamental for the continuous improvement of the material and the training events. All registered participants are encouraged to complete the whole survey, including the questions about sections that they did not attend or complete. To ensure a sufficient response rate to the surveys while maintaining anonymity, we implemented an anonymous verification process: submitting the survey generates a return code that can be entered into the Indico system to confirm that the survey was completed.

Figure 1 shows the pre-survey data collected during the registration for the training events from 360 registered participants between 2023 and 2025, providing insights into user familiarity with various Singularity/Apptainer commands and concepts. The significant majority of respondents had “Never heard of it” for most of the listed Apptainer commands and topics. This trend is consistent across all categories, indicating that most participants are researchers with no exposure to Apptainer, strongly motivating the need for training. Figure 2 illustrates the post-survey data collected from 82 participants after the training events. The plots show a visible increase in the knowledge of the students when compared with the pre-survey. Users generally reported being “I am familiar with” or “Very familiar” with core commands like apptainer pull, apptainer shell, and apptainer exec. This indicates a good grasp of fundamental operations. There’s a significant number of users who selected “Never heard of it” for certain commands, particularly ” apptainer instance”. This suggests the core learning objectives are met, while ‘apptainer instance’ serves as an advanced topic. Future iterations could explore participant interest in this feature more directly. On the other hand, it is important to remind that we encourage all event attendees, including those who did not start or complete this training module, to fill out the complete survey. This may explain the people who answered “Never heard of it” or “Used it once”.

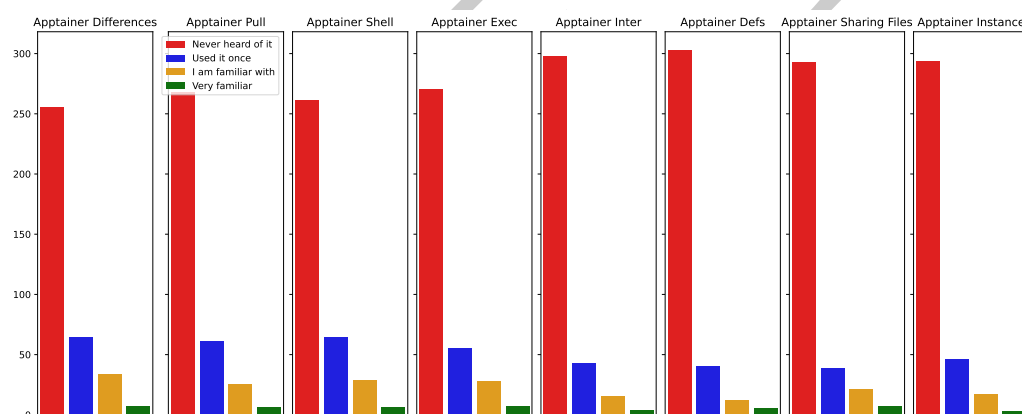


Figure 1: Pre-survey data collected during the training event registration, asking about familiarity with various Apptainer commands and concepts

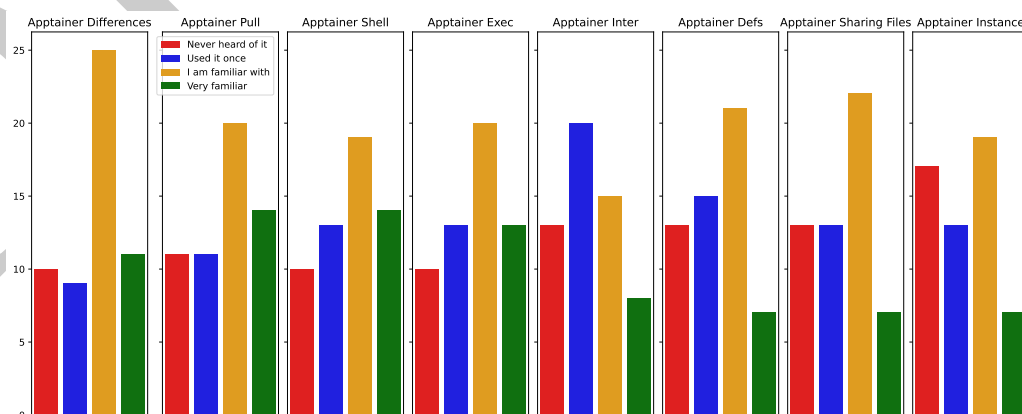


Figure 2: Post-survey data collected, asking about familiarity with various Apptainer commands and concepts after the training event

Trainees largely agreed that the material had a proper difficulty level and enough exercises to feel an interactive experience, as shown in the Figure 3. Similarly, most respondents

156 felt that the number of exercises was “About right” for interactive learning. This suggests
157 that the balance of theoretical content and practical application was effective.

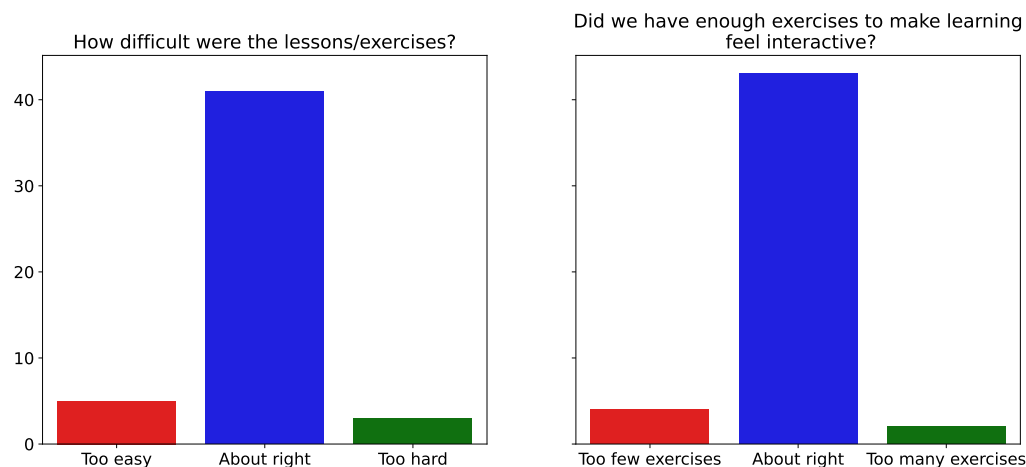


Figure 3: Metrics from the pre-event (post-event) surveys

158 Conclusions

159 This training module successfully addresses the need to introduce a reproducible, portable,
160 and resource-efficient data analysis in High Energy and Nuclear physics using Apptainer.
161 The module equips students and researchers with the skills to create and manage a
162 reproducible analysis environment, covering a range of concepts from the fundamentals of
163 containerization to the use of containers, the creation of images, and the best practices to
164 prepare and share a reproducible analysis.

165 The positive feedback consistently observed in the pre- and post-event surveys demonstrates
166 a significant increase in the knowledge of participants and satisfaction with the material's
167 difficulty and exercises. The demonstrated success of this module in various training events,
168 reaching 360 registered participants, illustrates its contribution to fostering a platform
169 for scientific collaboration and ensuring the long-term preservation of complex analyses,
170 independent of evolving environments.

171 Acknowledgements

172 We would like to thank all the members of our community in the HEP Software Foundation
173 and IRIS-HEP training for their voluntary contributions, big or small. We thank NSF
174 grants PHY-2323298 , OAC-1836650, OAC-1829707, and OAC-1829729 for support of the
175 training programs. This work was supported by the U.S. Department of Energy under
176 contract number DE-SC0012704.

177 References

- 178 Albrecht, J., & others. (2019). A Roadmap for HEP Software and Computing R&D for the
179 2020s. *Comput. Softw. Big Sci.*, 3(1), 7. <https://doi.org/10.1007/s41781-018-0018-8>
- 180 Antcheva, I., & others. (2009). ROOT: A C++ framework for petabyte data storage,
181 statistical analysis and visualization. *Comput. Phys. Commun.*, 180, 2499–2512.
182 <https://doi.org/10.1016/j.cpc.2009.08.005>

- 183 Bierlich, C., & others. (2022). A comprehensive guide to the physics and usage of PYTHIA
184 8.3. *SciPost Phys. Codeb.*, 2022, 8. <https://doi.org/10.21468/SciPostPhysCodeb.8>
- 185 Galli, M., Tejedor, E., & Wunsch, S. (2020). A New PyROOT: Modern, Interoperable
186 and More Pythonic. *EPJ Web Conf.*, 245, 06004. [https://doi.org/10.1051/epjconf/](https://doi.org/10.1051/epjconf/202024506004)
187 [202024506004](https://doi.org/10.1051/epjconf/202024506004)
- 188 IRIS-HEP. (n.d.). *Institute for Research and Innovation in Software for High Energy*
189 *Physics*. <http://iris-hep.org>.
- 190 Kurtzer, V. A. B., Gregory M. AND Sochat. (2017). Singularity: Scientific containers for
191 mobility of compute. *PLOS ONE*, 12(5), 1–20. [https://doi.org/10.1371/journal.pone.](https://doi.org/10.1371/journal.pone.0177459)
192 [0177459](https://doi.org/10.1371/journal.pone.0177459)
- 193 Pivarski, J., & others. (2017). *Uproot*. <https://doi.org/10.5281/zenodo.4340632>
- 194 Verkerke, W., & Kirkby, D. P. (2003). The RooFit toolkit for data modeling. *eConf*,
195 *C0303241*, MOLT007. <https://arxiv.org/abs/physics/0306116>

DRAFT